MAE 1001:
From numbers to loops: Unlocking Python's power

Introduction to Mechanical and Aerospace Engineering

Course website: https://gwu-mae1001.github.io/fall2025/

Prof. Kartik Bulusu, MAE Dept.

Graduate Teaching Assistant:
Preethi Siva Kumar (MAE Dept., MS student)

Learning Assistants:
Nathan Jannsen (Senior, MAE Dept.)
Olivia Falciani (Senior, MAE Dept.)
Shota Kakiuichi (Senior, MAE Dept.)

Tech support:
Rithik Gunuganti (CS Dept., MS student)

GW School of Engineering & Applied Science
THE GEORGE WASHINGTON UNIVERSITY

Fall 2025

Photo: Kartik Bulusu

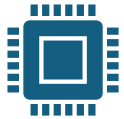
Sources:
https://numpy.org
https://www2.seas.gwu.edu/~cs4all/1012/unit2/module2.0.html
https://numpy.org/doc/stable/user/absolute_beginners.html

NumPy

```
import numpy as np
```

NumPy (**Numerical Python**) is an open source Python library that's used in almost every field of science and engineering.

Very large lists (million of elements or more) can slow down a program.



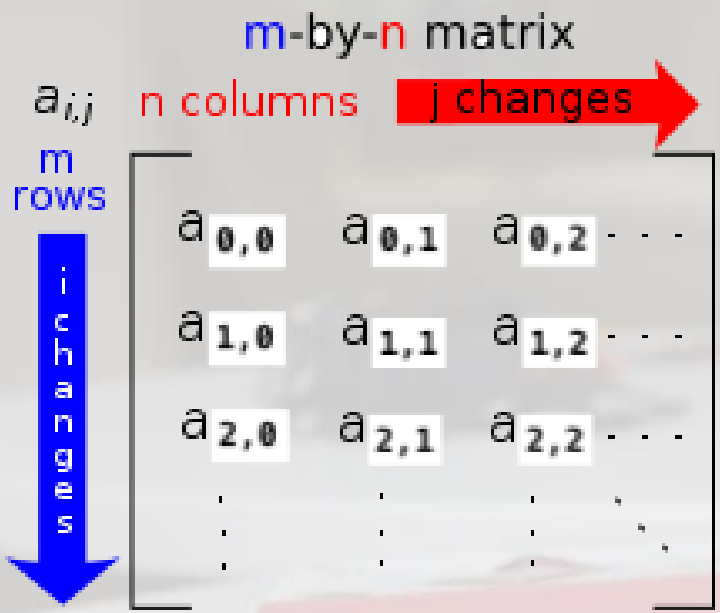
A list-of-lists is even slower for large sizes, and takes up a lot of memory.

NumPy arrays (we will often call them simply "arrays") were created as a separate structure in Python to enable efficient processing of lists of numbers, especially multidimensional lists.

```
>>> import numpy as np
>>> a = np.array([1, 2, 3])
```

```
>>> import numpy as np
>>> A = np.array([[-1, 2],[3, 4]])
>>> B1 = A * 6
>>> B2 = A * (1/6)
>>> len(B1)
>>> np.shape(B2)
```

Bookkeeping in a Matrix



Source:
http://en.wikipedia.org/wiki/Matrix_(mathematics)

Python:

```
>>> import numpy as np
>>> A = np.matrix([[-1, 2],[3, 4]])
>>> A[0,0]
>>> A[0,:]
>>> A[:,0]
>>> A[1,0]
```

python™

The ORDER of a matrix

- $A_{m\times n}$ *is* $m \times n$
- *Read as "m-by-n"*

a_{ij} *is called an ELEMENT*

- *at the* i^{th} *row and* j^{th} *column of A*

Photo: Kartik Bulusu

Matrix scalar operations

$$A = \begin{bmatrix} -1 & 2 \\ 3 & 4 \end{bmatrix}$$

$$s = 6$$

Scalar Multiplication and Division

- **Each element a_{ij}**
- *Is either **multiplied** with or **divided** by **s***

- Matrix, ***A*** has ***m*** ***rows*** and ***m*** ***columns***
- ***The ORDER of matrix, A ??***
- ***The ORDER of the scalar, s ??***

Python:

```
>>> import numpy as np
>>> A = np.matrix([[-1, 2],[3, 4]])
>>> B1 = A * 6
>>> B2 = A * (1/6)
>>> len(B1)
>>> np.shape(B2)
```

$$\begin{cases} A_{(m\times m)} * s_{(1\times 1)} = D_{(m\times m)} \\ A_{(m\times m)} * s^{-1}_{(1\times 1)} = F_{(m\times m)} \end{cases}$$

$$\begin{bmatrix} -1 & 2 \\ 3 & 4 \end{bmatrix} * 6 = \begin{bmatrix} -6 & 12 \\ 18 & 24 \end{bmatrix}$$

$$\begin{bmatrix} -1 & 2 \\ 3 & 4 \end{bmatrix} * \frac{1}{6} = \begin{bmatrix} -1/6 & 1/3 \\ 1/2 & -2/3 \end{bmatrix}$$

Photo: Kartik Bulusu

Python Commands:

```
>>> import numpy as np
>>> A = np.matrix([[-1, 2],[3, 4]])
>>> np.matrix('1 2; 3 4') # use Matlab-style syntax
>>> np.arange(25).reshape((5, 5)) # create a 1-d range and reshape
>>> np.array(range(25)).reshape((5, 5)) # pass a Python range and reshape
>>> np.array([5] * 25).reshape((5, 5)) # pass a Python list and reshape
>>> np.empty((5, 5)) # allocate, but don't initialize
>>> np.ones((5, 5)) # initialize with ones
>>> np.zeros([5, 5])
>>> np.ndarray((5, 5)) # use the low-level constructor
```

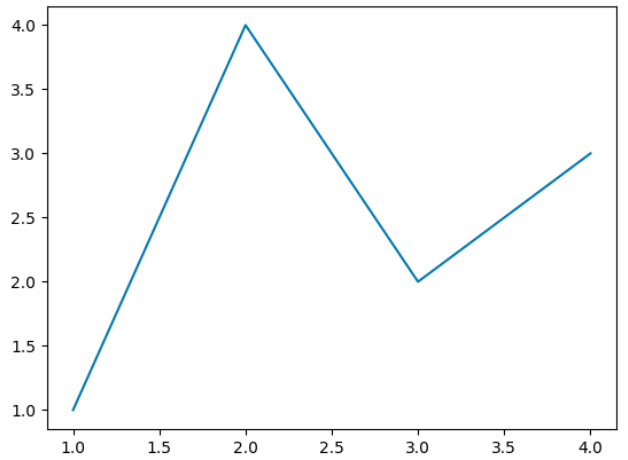
Sources:
https://matplotlib.org
https://www2.seas.gwu.edu/~cs4all/1012/unit2/module2.1.html

matplotlib

```
import matplotlib.pyplot as plt
```

Demo

```
1  import matplotlib.pyplot as plt
2  import numpy as np
3
4  # Create a figure containing a single axes.
5  fig, ax = plt.subplots()
6
7  # Plot some data on the axes.
8  ax.plot([1, 2, 3, 4], [1, 4, 2, 3])
9
10
11 plt.show()
```



Create publication quality plots.

Customize visual style and layout.

Export to many file formats.

Use a rich array of third-party packages built on Matplotlib.

Photo: Kartik Bulusu

Plotting data; the very basics

python™

matplotlib

https://matplotlib.org/stable/

```
import numpy as np
import matplotlib.pyplot as plt

plt.plot(x-values, y-values, 'go--', label='line 1', linewidth=2)

plt.grid()
plt.legend()
plt.show()
```

x-values and y-values
1D arrays, Row or column matrices or vectors containing the x- and y-coordinates of points on the graph.

'go-'
Color, marker and linestyle -> Green circles with a 'dashed' line
More marker and linestyle options:
https://matplotlib.org/stable/gallery/lines_bars_and_markers/linestyles.html
https://matplotlib.org/stable/api/markers_api.html

label=' '
Whatever text you put in between single quotes can be made to show in the plot legend

linewidth=
Whatever integer you enter will reflect the weight of the thickness of the line in your graph.

Programming pitfall: The two vector arguments x-values and y-values MUST have the same length.

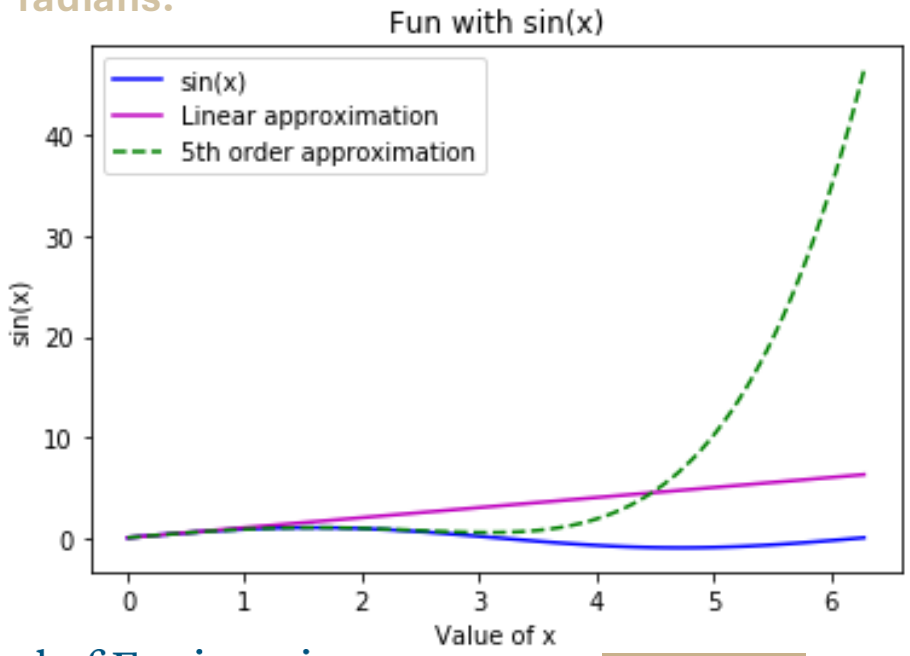
Photo: Kartik Bulusu

Plotting Example in Python

I have three functions:

$$y1 = \sin x$$
$$y2 = x$$
$$y3 = x - \frac{x^3}{3!} + \frac{x^5}{5!}$$

I would like to generate 100 values between 0 and 2π radians.




```
import numpy as np
import matplotlib.pyplot as plt
import math as mt
```

```
x = np.linspace(0,2*np.pi,100)
y1 = np.sin(x)
y2 = x;
y3 = x - (x**3/mt.factorial(3))+(x**5/mt.factorial(5))
```

```
# plt.figure()
plt.plot(x, y1, 'b', label='sin(x)')
plt.plot(x, y2, 'm', label='Linear approximation')
plt.plot(x, y3, 'g--', label='5th order approximation')
```

```
plt.xlabel('Value of x')
plt.ylabel('sin(x)')
plt.title('Fun with sin(x)')
```

I would like to plot three curves in one single plot !!

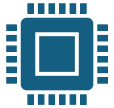
```
plt.legend()
plt.show()
```

Sources:
https://pandas.pydata.org
https://www2.seas.gwu.edu/~cs4all/1012/unit2/module2.1.html


```
import pandas as pd
```

Tools for **reading and writing data** between in-memory data structures and different formats: CSV and text files, Microsoft Excel, SQL databases, and the fast HDF5 format

High performance **merging and joining** of data sets



Intelligent **data alignment** and integrated handling of **missing data**: computations and easily manipulate messy data into an orderly form

Intelligent label-based **slicing, fancy indexing,** and **subsetting** of large data sets

Demo

```
import pandas as pd

x = pd.Series([1, 2, 4, 7])
print(x)
```

```
0    1
1    2
2    4
3    7
dtype: int64
```

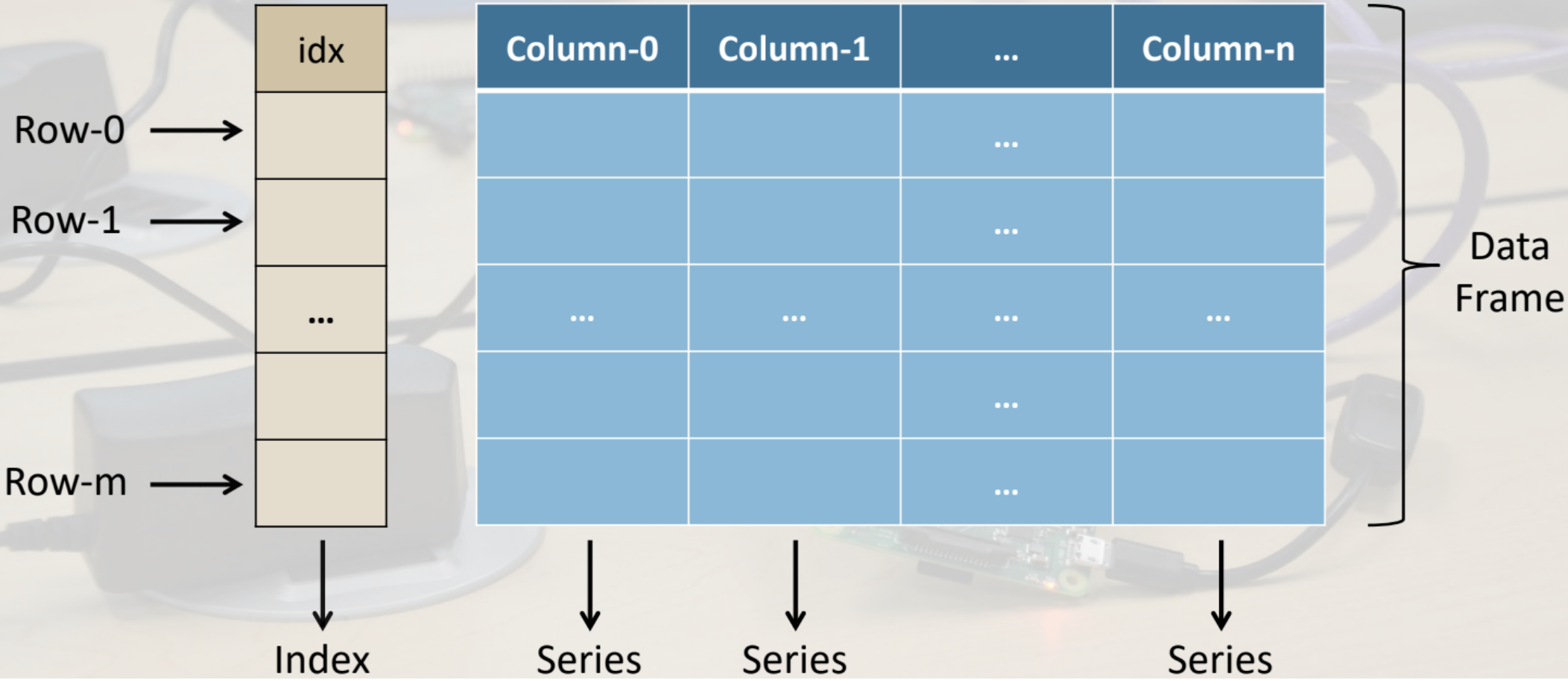
School of Engineering & Applied Science
THE GEORGE WASHINGTON UNIVERSITY

GW

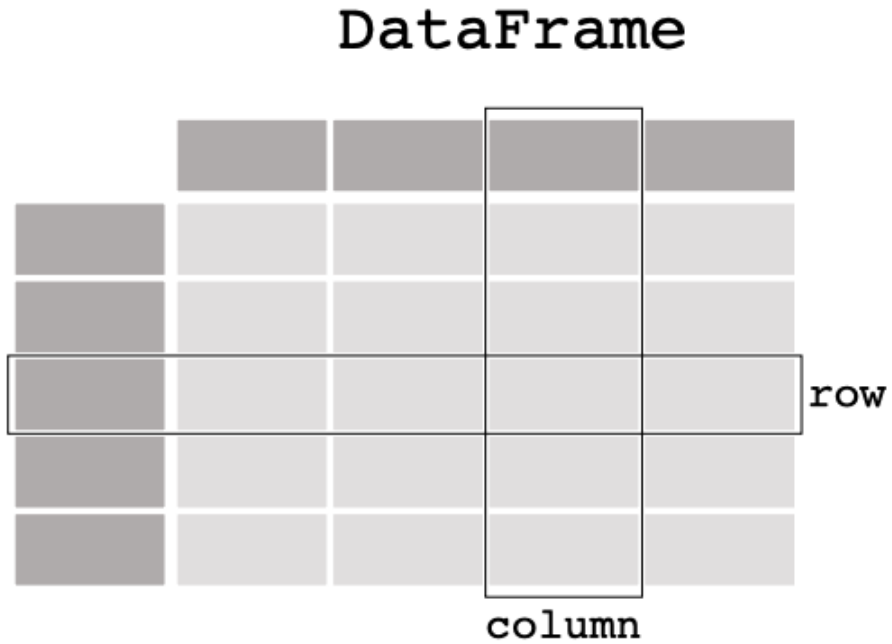
Typical Pandas Data Frame

Photo: Kartik Bulusu

```
import pandas as pd
df = pd.DataFrame();
print(df)
```



Sources:
https://pandas.pydata.org
https://www2.seas.gwu.edu/~cs4all/1012/unit2/module2.1.html



A DataFrame is a two-dimensional data structure, somewhat similar to a spreadsheet.

A DataFrame can represent more dimensions, but we will use it in two dimensions

A DataFrame consists of rows and columns.

Formally, a DataFrame is a collection of *columns*, each of which is a named Pandas Series.

```
import pandas as pd

hurricanes = {'name': ["Katrina", "Sandy", "Irene", "Ian"],
              'year':[2005, 2012, 2011, 2022],
              'wind speed': [280, 185, 195, 250],
              'damage': [122800, 67300, 15500,  112100]}

hurricane_df = pd.DataFrame(hurricanes)
print(hurricane_df)
```

```
      name  year  wind speed  damage
0  Katrina  2005         280  122800
1    Sandy  2012         185   67300
2    Irene  2011         195   15500
3      Ian  2022         250  112100
```

pd.DataFrame() was called on the dictionary of named lists to create the DataFrame.

```
print(hurricane_df['name'])
```

```
0    Katrina
1      Sandy
2      Irene
3        Ian
Name: name, dtype: object
```

DataFrame can be "indexed" directly by column.

Demo: Read data from a file into a DataFrame directly

"Study hard what interests you the most in the most undisciplined, irreverent and original manner possible."

— Richard Feynmann